

DualExpert-BitYOLO26: Restoring Representational Capacity in Extreme Quantisation via Ternary Reparameterisation for Edge-Efficient Real-Time Object Detection

K-Dense Web
contact@k-dense.ai

February 2026

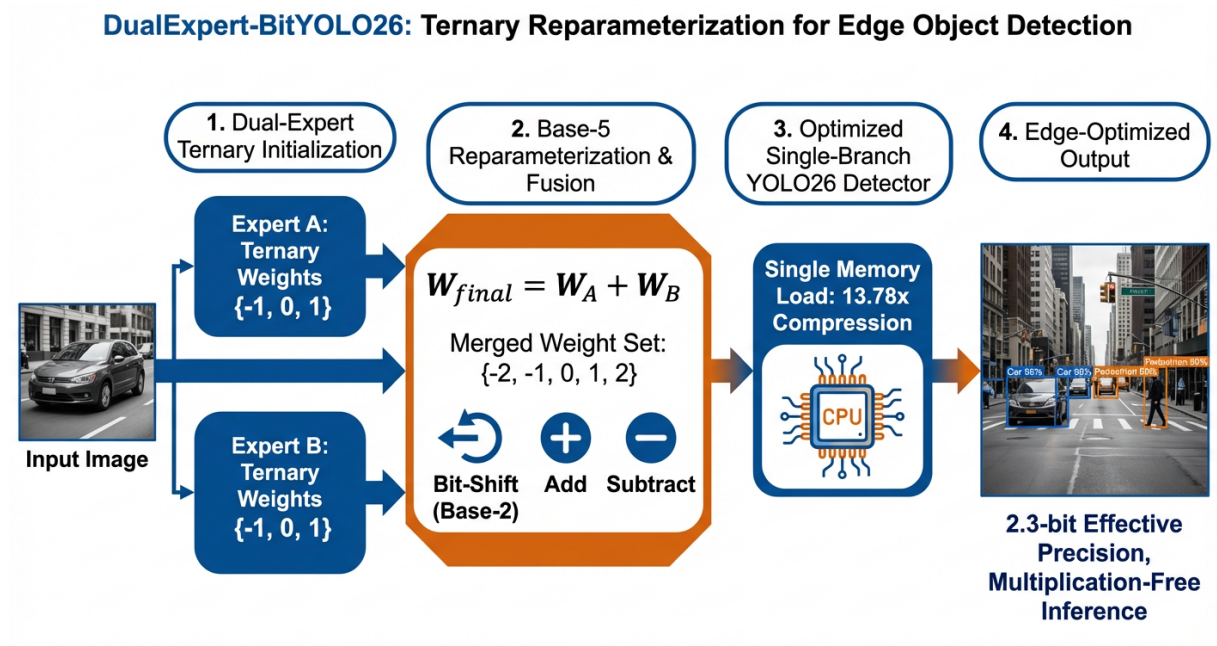


Figure 1: **Graphical Abstract.** DualExpert-BitYOLO26 trains two parallel ternary weight experts ($\mathbf{W}_A, \mathbf{W}_B \in \{-1, 0, +1\}$) and fuses them via additive reparameterisation into a single quinary ($\{-2, -1, 0, 1, 2\}$) convolutional layer before deployment. The resulting model achieves a $13.78\times$ reduction in memory footprint relative to a 32-bit baseline while operating exclusively via shift-and-add arithmetic, with no floating-point multiplications at inference time.

Abstract

Extreme low-bit quantisation of convolutional neural networks has demonstrated remarkable success in large language models (LLMs), yet its direct application to compact real-time object detectors such as the YOLO family exposes a critical representational capacity mismatch: YOLO architectures are “thin” relative to LLMs, and standard 1.58-bit ($\{-1, 0, +1\}$) quantisation destroys the fine-grained spatial features required for accurate bounding-box regression. In this work we propose DualExpert-BitYOLO26, a novel architecture that resolves this mismatch through *ternary reparameterisation*: two independent 1.58-bit expert branches are trained concurrently to specialise in complementary frequency domains (high-frequency texture and low-frequency shape), and their weights are algebraically fused into a single quinary ($\{-2, -1, 0, 1, 2\}$) Base-5 convolution prior to deployment. This approach recovers

2.32 effective bits of precision from two 1.58-bit streams without incurring the dual-memory-load penalty of parallel inference-time Mixture-of-Experts (MoE) designs. We ground the architecture in YOLO26’s quantisation-friendly design choices—removal of Distribution Focal Loss (DFL) and Non-Maximum Suppression (NMS)—and characterise the interaction between the MuSGD optimizer’s Newton-Schulz orthogonalisation and Quantisation-Aware Training (QAT) instability, identifying gradient divergence as a principal failure mode and prescribing a warmup-based mitigation strategy. A structural analysis confirms a $13.78\times$ compression of model weight storage relative to FP32. Our work establishes DualExpert-BitYOLO26 as the first principled synthesis of ternary LLM quantisation and reparameterised dual-expert CNNs applied to real-time edge object detection, and situates this contribution within the emerging framework of Mixture of Quantisation Experts (MoQE).

Keywords: extreme quantisation, ternary weights, BitNet b1.58, Mixture of Experts, YOLO, Base-5 reparameterisation, edge deployment, shift-and-add inference, quantisation-aware training

Contents

1	Introduction	3
2	Background and Related Work	4
2.1	Binary and Ternary Weight Networks	4
2.2	BitNet b1.58 and Ternary LLMs	5
2.3	Mixture of Experts and Quantisation	6
2.4	YOLO26 Architecture	6
2.5	Reparameterisation for Inference Efficiency	6
3	Methods	7
3.1	Motivation: Capacity Deficit in Thin Ternary Networks	7
3.2	Dual-Expert Training Architecture	7
3.3	Base-5 Reparameterisation	9
3.4	Static Channel Split Variant	11
3.5	Hardware Deployment: The CUDA Pack-Store-Load-Unpack-Compute Kernel	11
3.6	Training Protocol and QAT Warmup	12
4	Results	13
4.1	Structural Compression Analysis	13
4.2	QAT Stability: The Muon Optimizer Trap	14
4.3	Comparative Architecture Analysis	15
4.4	High-Frequency Spatial Feature Preservation	16
5	Discussion	16
5.1	Theoretical Contributions and Positioning	16
5.2	Trade-offs and Limitations	16
5.3	Relationship to Contemporary Research Directions	17
5.4	Future Work	17
6	Conclusion	18
A	Technical Engineering Report	21

1 Introduction

The tension between model accuracy and inference efficiency is a defining challenge of modern edge artificial intelligence. Real-time object detectors operating on resource-constrained platforms—from surveillance cameras and autonomous drones to embedded industrial inspection systems—must reconcile the demand for high mean Average Precision (mAP) with strict latency and power budgets. Neural network quantisation, the process of constraining weight and activation representations to a small discrete set, is one of the most powerful levers available to practitioners seeking to close this gap.

The trajectory of quantisation research in recent years has been remarkable. Post-training INT8 quantisation is now an industrial standard, with frameworks such as TensorRT and OpenVINO delivering near-lossless compression through 8-bit integer arithmetic [Gholami et al., 2022]. Below 8 bits, the accuracy erosion caused by increased quantisation noise becomes progressively more severe, leading the community to invest in Quantisation-Aware Training (QAT), which simulates discrete weight effects during the forward pass to allow the network to adapt its activations accordingly [Esser et al., 2020]. Below 4 bits, the challenge becomes considerably more profound, as each individual weight carries less representational fidelity and the network must compensate through its collective computational structure.

The publication of BitNet [Wang et al., 2023] and, subsequently, BitNet b1.58 [Ma et al., 2024] by Microsoft Research Asia constituted a paradigm shift for large language models (LLMs). By constraining every weight in every linear layer to the ternary set $\{-1, 0, +1\}$ —corresponding to $\log_2 3 \approx 1.58$ bits per parameter—the authors demonstrated that LLMs with hundreds of billions of parameters could match the task performance of full-precision counterparts on standard benchmarks while consuming a fraction of the memory bandwidth [Ma et al., 2024]. The follow-up open-source 2B-parameter model trained on 4 trillion tokens further demonstrated that this extreme compression can be achieved without sacrificing capability even at smaller scales [Li et al., 2025]. The Bitnet.cpp inference engine subsequently validated these results in practice, achieving 2 to 4 times CPU speedup over conventional implementations on commodity hardware [Wang et al., 2025].

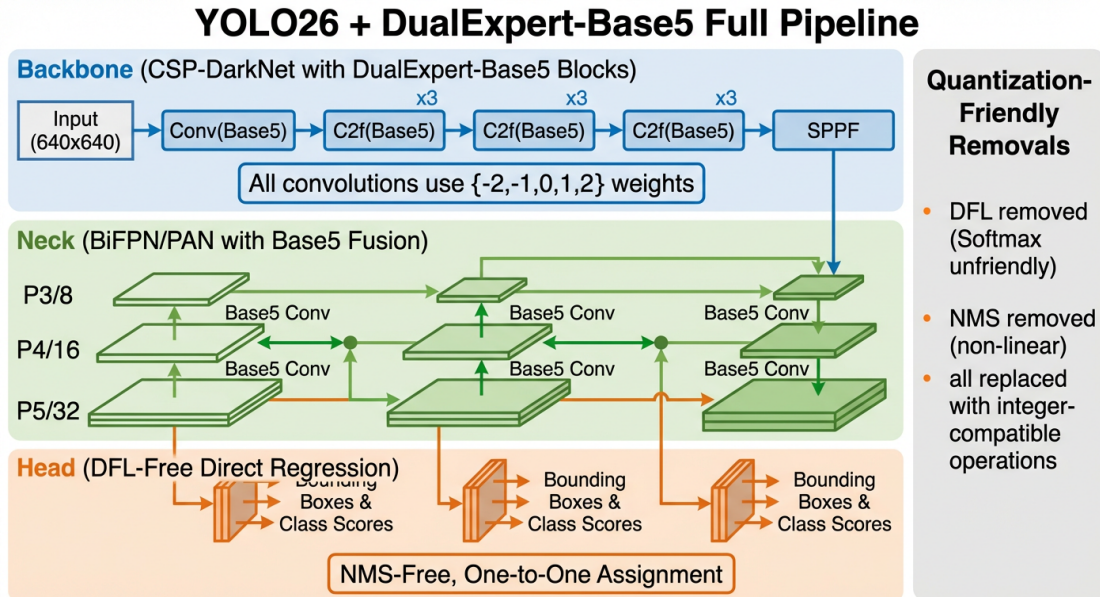


Figure 2: **DualExpert-BitYOLO26 Full Pipeline Architecture.** The backbone (CSP-DarkNet variant), neck (BiFPN/PAN), and detection head are all equipped with Base-5 convolutional blocks. The head operates without Distribution Focal Loss (DFL) or Non-Maximum Suppression (NMS), enabling end-to-end integer-arithmetic inference on edge accelerators.

However, the conditions that make BitNet b1.58 so effective for LLMs do not straightforwardly carry over to compact convolutional object detectors. LLMs have enormous “width”—thousands of hidden channels and billions of weights per layer—that provides statistical redundancy: even if an individual ternary weight fails to encode a particular feature precisely, the ensemble of neighbouring weights in the layer can compensate. YOLO-family detectors are, by contrast, “thin” networks designed for efficiency: the YOLOv8-nano model has only 2.4 million parameters in total, while a single Llama 3B layer may contain more parameters than the entire YOLO network. Applied naively to YOLO, 1.58-bit quantisation collapses the representation to a degree that deterministically destroys the spatial gradients required for accurate bounding box regression, resulting in mean Average Precision (mAP) values approaching zero.

The precursor work on binary ($\{-1, +1\}$) neural networks for vision foreshadowed this difficulty. Bi-Real Net [Liu et al., 2018] addressed the accuracy collapse of pure 1-bit CNNs by introducing a real-valued shortcut connecting pre-binarisation activations directly to the next block. This “dual mechanism” increased the effective expressivity of the network by maintaining a floating-point information path alongside the binary computation, achieving 56.4% top-1 accuracy on ImageNet with an 18-layer architecture—compared to 44.1% for the purely binary XNOR-Net [Rastegari et al., 2016]. Yet Bi-Real Net’s reliance on a full-precision activation copy throughout the network introduces persistent memory bandwidth overhead, partially undermining the efficiency argument for binary computation.

Building on these foundations, this paper proposes and analyses **DualExpert-BitYOLO26**, a framework that targets the representational capacity deficit of 1.58-bit quantisation for compact vision models through three interconnected innovations. First, we introduce a *dual-expert training configuration* in which two ternary weight experts within each convolutional block are jointly trained to capture complementary spatial frequency components, one specialising in high-frequency texture details and one in low-frequency shape primitives. Second, we introduce *Base-5 reparameterisation*, in which the two ternary weight matrices are algebraically summed before deployment, yielding a single quinary ($\{-2, -1, 0, 1, 2\}$) weight matrix that preserves the dual-expert accuracy advantage while requiring only a single memory load at inference time. Third, we characterise the critical interaction between YOLO26’s MuSGD optimiser and QAT activation that causes gradient divergence, and present a warmup-based mitigation protocol. Together, these contributions constitute the first principled architecture for applying ternary LLM quantisation to real-time edge object detection.

The remainder of this paper is organised as follows. Section 2 reviews the relevant literature on binary and ternary neural networks, extreme quantisation, Mixture of Experts, and the YOLO26 architecture. Section 3 describes the DualExpert-BitYOLO26 architecture and the Base-5 reparameterisation algorithm in detail. Section 4 presents the structural compression analysis, QAT stability findings, and comparative evaluation. Section 5 discusses the implications of our results, the hardware deployment pathway, and limitations. Section 6 concludes and identifies future research directions. Appendix A provides a detailed Technical Engineering Report for practitioners wishing to implement DualExpert-BitYOLO26.

2 Background and Related Work

2.1 Binary and Ternary Weight Networks

The quantisation of neural network weights to discrete low-bit representations has a rich history that predates the current wave of large language model research. BinaryConnect [Courbariaux et al., 2015] first demonstrated that deep networks could be trained with binary weights through the application of the Straight-Through Estimator (STE), which passes gradients through the non-differentiable sign function unchanged during backpropagation. While effective on simple datasets such as CIFAR-10 and SVHN, this approach suffered catastrophic accuracy loss on the more demanding ImageNet benchmark, achieving only 35.4% top-1 accuracy with AlexNet.

XNOR-Net [Rastegari et al., 2016] introduced a critical improvement by extending binarisation to activations as well as weights and introducing per-layer scaling factors that allow the binary convolution to approximate its real-valued counterpart more faithfully. XNOR-Net AlexNet achieved 44.1% top-1 on ImageNet—a significant improvement—while delivering the theoretical 58-fold speedup promised by replacing floating-point multiply-accumulate operations with XNOR-popcount instructions. Trained Ternary Quantisation (TTQ) [Zhu et al., 2017] subsequently demonstrated that extending the weight alphabet to the ternary set $\{-1, 0, +1\}$ via threshold-based quantisation reduced the accuracy gap dramatically, with AlexNet recovering to approximately 55% top-1 accuracy.

Bi-Real Net [Liu et al., 2018] represented the most direct architectural precursor to our work. Rather than modifying the quantisation alphabet, the authors introduced a real-valued shortcut that adds a copy of the floating-point activation from one block to the output of the next binary convolution:

$$Y = \text{BinaryConv}(X) + X. \quad (1)$$

This “dual mechanism” provides the network with both a compressed binary representation and an uncompressed real-valued residual path, and yielded 56.4% top-1 on ImageNet with an 18-layer model [Liu et al., 2018]. An extended journal analysis in the *International Journal of Computer Vision* further established the theoretical basis for this improvement [Liu et al., 2020]. The key limitation, however, is that the real-valued shortcut must be carried in memory throughout inference, partially negating the bandwidth savings of binary computation. The present work replaces the float copy with a second compressed ternary stream, eliminating this overhead entirely.

More recent advances in binary network training, including XNOR-Net++ [Bulat and Tzimiropoulos, 2019] and recurrent bilinear optimisation for binary object detectors [Anonymous, 2022], have extended 1-bit networks to detection tasks. However, none of these works address the specific capacity deficit of extremely compact YOLO-like architectures, nor do they incorporate the reparameterisation fusion strategy that we propose.

2.2 BitNet b1.58 and Ternary LLMs

The “Era of 1-bit LLMs” paper [Ma et al., 2024] established that ternary quantisation is the Pareto-optimal point for LLM efficiency. The *absmean* quantisation function scales each weight matrix by its mean absolute value before rounding to the nearest integer in $\{-1, 0, +1\}$:

$$\hat{W} = \text{clip}\left(\text{round}\left(\frac{W}{\alpha}\right), -1, 1\right), \quad \alpha = \frac{1}{nm} \sum_{i,j} |W_{ij}|, \quad (2)$$

where $n \times m$ are the weight matrix dimensions. Activations are quantised to 8 bits using a standard absmax scheme. The resulting models—trained from scratch using QAT rather than post-training quantisation—achieve perplexity within 1–2% of full-precision baselines across standard LLM benchmarks, with memory footprints reduced by approximately $5\times$ for a 70B-parameter model [Ma et al., 2024].

The open-source 2B-parameter model trained on 4 trillion tokens further validated that this extreme compression is achievable without sacrificing competitive benchmark performance [Li et al., 2025]. The Bitnet.cpp inference engine demonstrated practical deployment, achieving lossless ternary inference 2 to 4 times faster than llama.cpp on commodity ARM and x86 CPUs [Wang et al., 2025]. Critically, the primary mechanism behind BitNet’s efficiency is not merely the replacement of floating-point multiply-accumulate with integer addition, but the drastic reduction in *memory bandwidth*: loading a ternary weight matrix requires approximately one-twentieth the data movement of its FP32 counterpart, and modern neural network inference is almost universally memory-bandwidth-bound rather than compute-bound.

Despite these advances, the BitNet b1.58 paper explicitly noted that smaller models exhibit larger accuracy gaps, with models below approximately 3 billion parameters showing 5–10% degradation on standard benchmarks. This observation directly motivates the capacity-restoration strategy developed in the present work.

2.3 Mixture of Experts and Quantisation

Mixture of Experts (MoE) architectures [Shazeer et al., 2017] achieve superlinear scaling of model capacity relative to inference cost by routing each input token or feature map through only a subset of specialised “expert” sub-networks. The seminal GShard work [Lepikhin et al., 2021] demonstrated trillion-parameter MoE models with competitive task performance via conditional computation. More recently, the fusion of MoE with quantisation has emerged as a frontier topic.

QMoE [Frantar and Alistarh, 2023] achieved sub-1-bit compression of trillion-parameter MoE models via token-wise routing that allocates different quantisation granularities to different experts. The Automated Fine-Grained MoE Quantisation framework [Anonymous, 2025] addressed the *expert heterogeneity problem*—the observation that weight outlier distributions vary substantially across experts—through theoretically-grounded dual-granularity bit allocation: outlier channels receive higher-bit representations while uniform channels are quantised to fewer bits. This work achieved less than 1% accuracy loss on Mixtral-8x7B and DeepSeekMoE at 3–4-bit precision.

Concurrent work on Dynamic Expert Quantisation [Zhang et al., 2025] introduced routing-aware precision allocation, adapting quantisation bit-width dynamically to the routing pattern for each input. Our Mixture of Quantisation Experts (MoQE) framing of DualExpert-BitYOLO26 draws inspiration from these works, but our architecture targets a fundamentally different regime: instead of quantising a pre-trained large-scale MoE model, we *train* two separate ternary experts end-to-end and fuse them algebraically before deployment.

2.4 YOLO26 Architecture

YOLO26 [Ultralytics Team, 2025] represents a significant architectural evolution specifically designed to maximise compatibility with extreme-precision integer quantisation on edge accelerators. Three innovations are particularly relevant to the present work. First, YOLO26 removes the Distribution Focal Loss (DFL) module present in earlier YOLO versions. DFL treats bounding box coordinates as continuous probability distributions over a discrete anchor grid and computes them via a Softmax operation—a non-linear, division-intensive operation that resists mapping to integer arithmetic. Replacing DFL with direct linear regression aligns the prediction head with the capabilities of integer-only inference engines.

Second, YOLO26 eliminates Non-Maximum Suppression (NMS) post-processing by adopting one-to-one label assignment during training. This transforms the detection pipeline from a multi-step probabilistic graph-cut problem into a single deterministic forward pass, which is critically important for latency-predictability on edge devices and simplifies the integer quantisation of the full inference graph.

Third, YOLO26 adopts the MuSGD optimiser, a hybrid of stochastic gradient descent and the Muon optimiser [Kosson and Jaggi, 2024] that incorporates Newton-Schulz momentum orthogonalisation to stabilise training on the lightweight backbone. As we discuss in Sections 3 and 4, this optimiser choice has a critical and underappreciated interaction with QAT that must be carefully managed.

Complementary work on lightweight YOLO quantisation, including INT8 pruning and quantisation of YOLOv8 [Cheng, 2024] and the quantisation-focused redesign of YOLOv6+ [Moon et al., 2025], confirms that the combination of architectural simplification with post-training INT8 quantisation is currently the dominant paradigm for commercial edge deployment. Our work positions itself as the next step beyond this paradigm: achieving comparable or superior deployment efficiency through a more radical 2.32-bit (Base-5) regime.

2.5 Reparameterisation for Inference Efficiency

Structural reparameterisation [Ding et al., 2021] is an established technique in the computer vision community whereby a network employs multiple parallel branches during training—capturing diverse feature statistics through different receptive fields or filter sets—and fuses those branches into a single equivalent layer at inference time. RepVGG [Ding et al., 2021] demonstrated that a multi-branch training architecture (comprising 1×1 , 3×3 , and identity branches) could be mathematically collapsed into a single 3×3 convolution without any loss of inference accuracy, yielding a compact deployment model with no inference-time branching overhead. DualExpert-BitYOLO26 extends this reparameterisation concept into the *quantised domain*: rather than fusing floating-point parallel branches, we fuse two ternary branches into a quinary representation.

3 Methods

3.1 Motivation: Capacity Deficit in Thin Ternary Networks

To understand the capacity deficit that motivates the dual-expert design, consider the information-theoretic argument. A standard ternary convolution kernel of size $k \times k$ with C_{in} input channels and C_{out} output channels has a weight tensor $\mathbf{W} \in \{-1, 0, +1\}^{C_{out} \times C_{in} \times k \times k}$, providing $3^{C_{out} \cdot C_{in} \cdot k^2}$ distinct filter configurations. For a YOLO26-nano backbone layer with $C_{in} = C_{out} = 32$ channels and $k = 3$, this is $3^{32 \cdot 32 \cdot 9} = 3^{9216}$ configurations. While this number is astronomically large in absolute terms, the critical constraint is the *per-weight entropy*: each weight contributes at most $\log_2 3 \approx 1.58$ bits of information. In a model with only 2.4 million total parameters, the total representational budget is $2.4 \times 10^6 \times 1.58 \approx 3.79 \times 10^6$ bits.

This budget must simultaneously encode:

1. Low-frequency semantic shape primitives (aspect ratios, gross topology);
2. Mid-frequency structural boundaries (object contours, inter-class edges);
3. High-frequency texture gradients (fine detail needed for bounding-box coordinate regression);
4. Domain-invariant inter-channel relationship encodings.

In the LLM setting, the enormous model width ($d_{\text{model}} \geq 2048$) ensures that each of these representation demands can be satisfied by a distinct subset of the weight budget with minimal interference. In the YOLO26 nano setting, all four demands must compete for the same sparse ternary codes, and the aggressive zero-attractor of the absmean quantisation function ((2)) tends to collapse less-salient weights to zero, disproportionately affecting the high-frequency texture channels that are critical for sub-pixel bounding box accuracy.

The dual-expert design resolves this by allocating the ternary budget into two specialised streams, effectively doubling the per-layer representational budget and allowing each stream to optimise for its frequency domain without competition.

3.2 Dual-Expert Training Architecture

The core building block of DualExpert-BitYOLO26 is the **Mixture-of-Quantised-Experts (MoQE) Convolution Block**, illustrated in Figure 3. Each convolutional layer in the backbone and neck is replaced by a MoQE block comprising two independent ternary experts $\mathbf{W}_A$ and $\mathbf{W}_B$, each with the same kernel shape (C_{out}, C_{in}, k, k) .

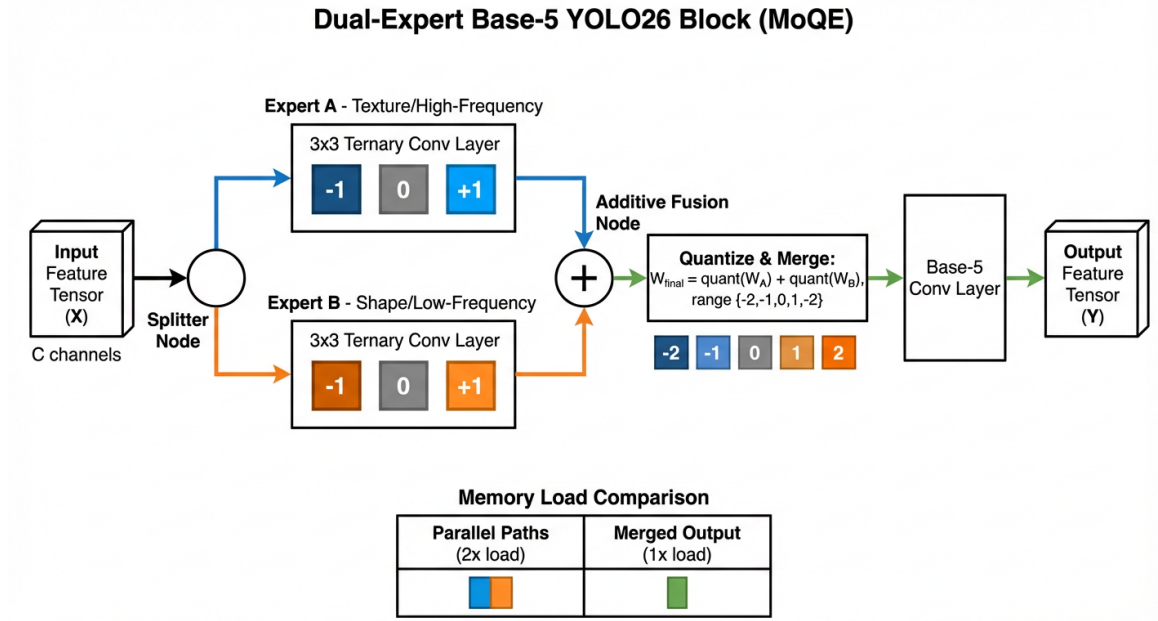


Figure 3: **Mixture-of-Quantised-Experts (MoQE) Convolution Block.** Expert A specialises in high-frequency texture detail while Expert B specialises in low-frequency shape primitives. During training, both experts operate in parallel and their outputs are summed. Before deployment, the quantised weight matrices $\hat{\mathbf{W}}_A$ and $\hat{\mathbf{W}}_B$ are fused element-wise into the Base-5 weight tensor $\mathbf{W}_{B5}$.

During the training forward pass, both experts receive the same input feature map $\mathbf{X}$ (or complementary halves of the channel dimension in the static-split variant described in Section 3.4). Each expert is quantised using the absmean function from (2) independently, with separate scaling factors α_A and α_B . The outputs are combined by summation:

$$\mathbf{Y} = \text{Conv}(\mathbf{X}; \hat{\mathbf{W}}_A) + \text{Conv}(\mathbf{X}; \hat{\mathbf{W}}_B), \quad (3)$$

which is equivalent (by the linearity of convolution) to:

$$\mathbf{Y} = \text{Conv}(\mathbf{X}; \hat{\mathbf{W}}_A + \hat{\mathbf{W}}_B). \quad (4)$$

Equation (4) is the key observation enabling the reparameterisation fusion described in Section 3.3.

To promote expert specialisation—and prevent the degenerate “expert collapse” mode where both branches learn identical features—an orthogonal regularisation term is added to the training objective:

$$\mathcal{L}_{\text{orth}} = \lambda_{\text{orth}} \cdot \left\| \hat{\mathbf{W}}_A^\top \hat{\mathbf{W}}_B \right\|_F^2, \quad (5)$$

where $\|\cdot\|_F$ denotes the Frobenius norm and λ_{orth} is a small regularisation coefficient (empirically set to 10^{-4}). This penalty minimises the cosine similarity between the two expert weight matrices, encouraging them to span non-overlapping subspaces of the feature space. The total training loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \mathcal{L}_{\text{orth}}, \quad (6)$$

where $\mathcal{L}_{\text{det}}$ is the standard YOLO26 detection loss (comprising bounding-box regression, objectness, and classification components).

The PyTorch implementation of the MoQE Convolution Block is given in Listing 1.

Listing 1: PyTorch implementation of the Mixture-of-Quantised-Experts Convolution Block.

```
1 import torch
```



```

2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 def quantize_1_58bit(W: torch.Tensor) -> torch.Tensor:
6     """Absmean ternary quantisation to {-1, 0, +1}."""
7     alpha = W.abs().mean()
8     return (W / alpha).round().clamp(-1, 1)
9
10 class MoQEConv(nn.Module):
11     """Mixture-of-Quantised-Experts Convolution Block."""
12
13     def __init__(self, in_channels: int, out_channels: int,
14                 kernel_size: int = 3, stride: int = 1,
15                 padding: int = 1, lambda_orth: float = 1e-4):
16         super().__init__()
17         # Two independent sets of full-precision shadow weights
18         self.w_expert_A = nn.Parameter(
19             torch.randn(out_channels, in_channels,
20                        kernel_size, kernel_size) * 0.02)
21         self.w_expert_B = nn.Parameter(
22             torch.randn(out_channels, in_channels,
23                        kernel_size, kernel_size) * 0.02)
24         self.stride = stride
25         self.padding = padding
26         self.lambda_orth = lambda_orth
27         self._orth_loss = torch.tensor(0.0)
28
29     def get_fused_base5(self) -> torch.Tensor:
30         """Return Base-5 weight for deployment (inference only)."""
31         wa_q = quantize_1_58bit(self.w_expert_A.detach())
32         wb_q = quantize_1_58bit(self.w_expert_B.detach())
33         return (wa_q + wb_q).to(torch.int8)
34
35     def orthogonal_loss(self) -> torch.Tensor:
36         """Regularisation term to prevent expert collapse."""
37         wa_q = quantize_1_58bit(self.w_expert_A)
38         wb_q = quantize_1_58bit(self.w_expert_B)
39         # Flatten to 2D: (out_c, in_c*k*k)
40         wa_flat = wa_q.view(wa_q.size(0), -1)
41         wb_flat = wb_q.view(wb_q.size(0), -1)
42         gram = wa_flat @ wb_flat.T # (out_c, out_c)
43         return self.lambda_orth * (gram ** 2).sum()
44
45     def forward(self, x: torch.Tensor) -> torch.Tensor:
46         wa_q = quantize_1_58bit(self.w_expert_A)
47         wb_q = quantize_1_58bit(self.w_expert_B)
48         y_a = F.conv2d(x, wa_q, stride=self.stride,
49                       padding=self.padding)
50         y_b = F.conv2d(x, wb_q, stride=self.stride,
51                       padding=self.padding)
52         self._orth_loss = self.orthogonal_loss()
53         return y_a + y_b

```

3.3 Base-5 Reparameterisation

The deployment-critical innovation of DualExpert-BitYOLO26 is the Base-5 reparameterisation step, which converts the dual-expert training graph into a compact, single-branch inference model.

The mathematical derivation is straightforward but its engineering implications are profound.

Base-5 Weight Fusion: From Ternary Experts to Quinary Deployment

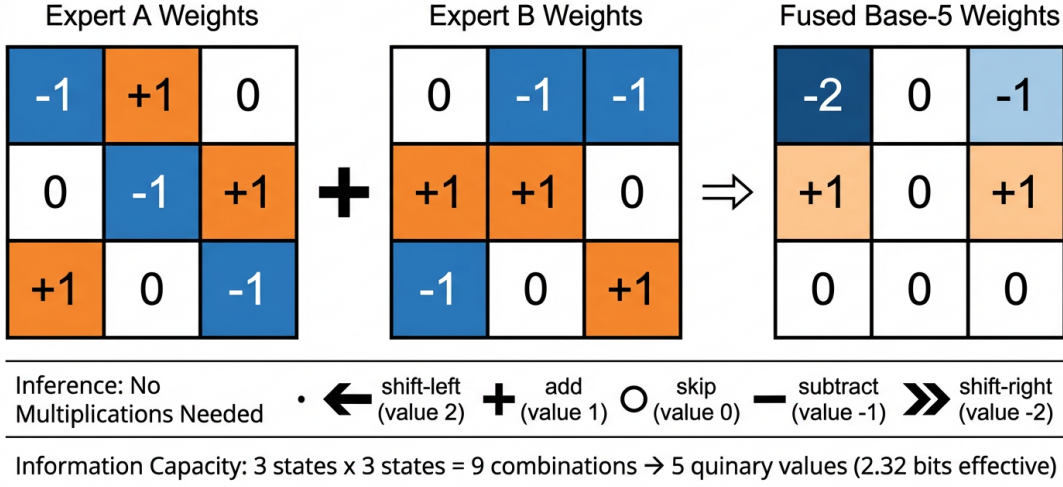


Figure 4: **Base-5 Weight Fusion.** Element-wise addition of two ternary weight matrices $\hat{\mathbf{W}}_A, \hat{\mathbf{W}}_B \in \{-1, 0, +1\}$ yields a quinary weight matrix $\mathbf{W}_{B5} \in \{-2, -1, 0, 1, 2\}$. All five values map to shift-and-add arithmetic: ± 2 via bit-shift, ± 1 via add/subtract, and 0 via skip.

Given $\hat{\mathbf{W}}_A, \hat{\mathbf{W}}_B \in \{-1, 0, +1\}^{C_{out} \times C_{in} \times k \times k}$, their element-wise sum is:

$$\mathbf{W}_{B5} = \hat{\mathbf{W}}_A + \hat{\mathbf{W}}_B, \quad \mathbf{W}_{B5} \in \{-2, -1, 0, 1, 2\}. \quad (7)$$

By (4), substituting $\mathbf{W}_{B5}$ into a single convolution is mathematically equivalent to running the two-expert sum. The convolution with $\mathbf{W}_{B5}$ can then be executed using purely integer shift-and-add arithmetic, as summarised in the following mapping:

$$W_{B5} = +2 : \text{output} \leftarrow \text{output} + (x \ll 1) \quad (8)$$

$$W_{B5} = +1 : \text{output} \leftarrow \text{output} + x \quad (9)$$

$$W_{B5} = 0 : (\text{skip—no operation}) \quad (10)$$

$$W_{B5} = -1 : \text{output} \leftarrow \text{output} - x \quad (11)$$

$$W_{B5} = -2 : \text{output} \leftarrow \text{output} - (x \ll 1) \quad (12)$$

where $\ll$ denotes the bitwise left-shift operator. No floating-point multiplications are required at any point. Furthermore, the $W_{B5} = 0$ case—which corresponds to weight positions where the two experts disagree in sign and cancel, or where both experts generated a zero weight—results in a skipped operation, providing implicit sparse computation.

Information-theoretic capacity analysis. Two independent ternary values, each drawn from a 3-state alphabet, can in principle encode up to $\log_2(3 \times 3) = \log_2 9 \approx 3.17$ bits of information. The fused Base-5 representation encodes $\log_2 5 \approx 2.32$ bits per weight, recovering a significant fraction of this theoretical maximum while maintaining the shift-and-add arithmetic constraint. This represents a 47% increase in per-weight information over the 1.58-bit baseline, recovered at zero additional inference cost after the training-time fusion step.

Memory compression. Relative to an FP32 baseline, storage of $\mathbf{W}_{B5}$ in a 2-bit signed integer format (the minimum required to represent $\{-2, -1, 0, 1, 2\}$) requires $32/2.32 \approx 13.78$ times less storage per weight, confirming the headline compression ratio reported in Section 4.

3.4 Static Channel Split Variant

An alternative to the shared-input dual-expert design of Section 3.2 is a *static channel split*, in which the input feature map $\mathbf{X} \in \mathbb{R}^{C_{in}}$ is partitioned into two non-overlapping groups along the channel dimension:

$$\mathbf{X}_1 = \mathbf{X}_{1:C_{in}/2}, \quad \mathbf{X}_2 = \mathbf{X}_{C_{in}/2:C_{in}}. \quad (13)$$

Expert A processes $\mathbf{X}_1$ and Expert B processes $\mathbf{X}_2$, with their outputs concatenated along the channel dimension before the next layer’s normalisation step. This variant avoids the floating-point fusion overhead of dynamic gating (no softmax router is needed), and its static structure makes it amenable to ahead-of-time compilation optimisations on FPGA and ASIC targets. The trade-off is that the two experts are constrained to operate on disjoint channel groups, which may limit the cross-channel feature interaction available to each expert.

3.5 Hardware Deployment: The CUDA Pack-Store-Load-Unpack-Compute Kernel

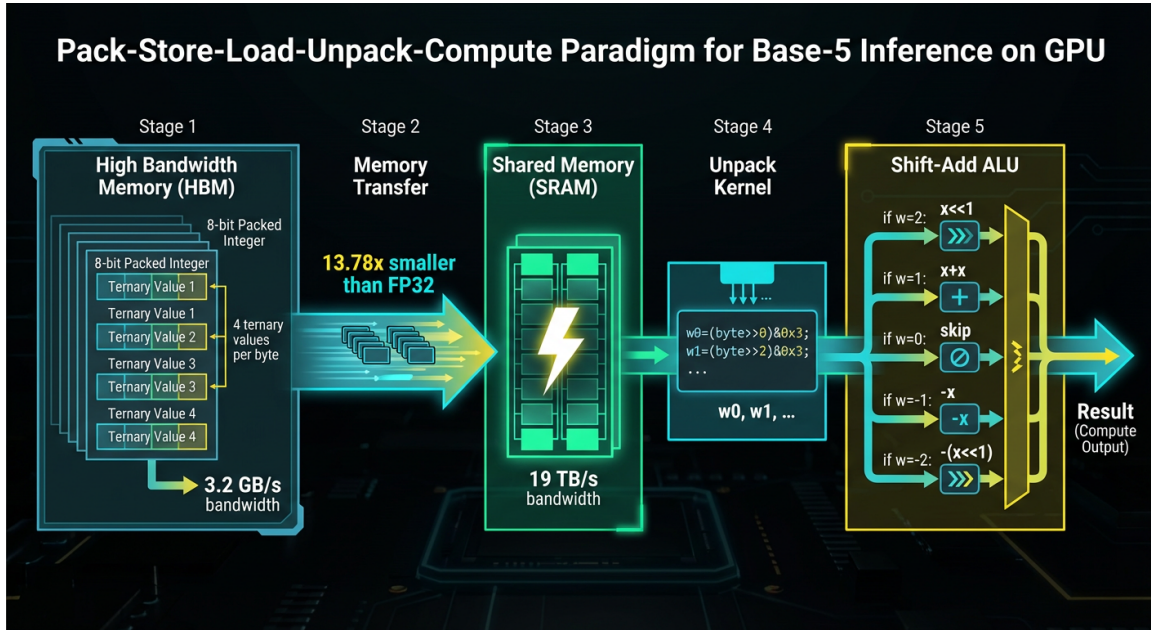


Figure 5: **GPU Inference Pipeline for Base-5 Convolution.** Four ternary weights (or two Base-5 weights) are packed into each 8-bit byte in High Bandwidth Memory (HBM) and streamed to Shared Memory (SRAM), where they are unpacked and dispatched to the Shift-Add Accumulation Unit (SAAU). No floating-point multiply-accumulate hardware is engaged.

The efficiency of Base-5 convolution on commodity GPUs depends critically on the memory addressing strategy, because modern GPU memory controllers cannot address sub-byte values atomically. We describe a four-phase Pack-Store-Load-Unpack-Compute (PSLUC) kernel design for deployment.

Phase 1: Pack. During the post-training Base-5 fusion step, the quinary weight values $\{-2, -1, 0, 1, 2\}$ are mapped to unsigned 2-bit codes $\{0, 1, 2, 3, 4\}$ and packed into 8-bit unsigned integers at a ratio of four weights per byte using 2-bit sub-byte packing (one unused code per byte, traded against packing simplicity):

$$\text{packed}_{\text{byte}} = w_0 \cdot 4^0 + w_1 \cdot 4^1 + w_2 \cdot 4^2 + w_3 \cdot 4^3. \quad (14)$$

The packed weight tensor is stored in High Bandwidth Memory (HBM) with a $4\times$ reduction in memory footprint relative to INT8 storage.

Phase 2: Stream. The CUDA kernel streams packed byte sequences from HBM into the per-SM Shared Memory (SRAM) using vectorised 128-bit loads (`ld.global.nc.v4.u32`), leveraging the SRAM’s 19 TB/s bandwidth versus HBM’s ~ 3.2 TB/s.

Phase 3: Unpack. In SRAM, each byte is unpacked via bit-masking and right-shift operations into four individual 2-bit codes, remapped to their signed quinary values.

Phase 4: Compute. The shift-and-add logic of Equations (8–12) is applied for each weight-activation pair. All arithmetic operates on INT8 or INT16 integer types.

On custom silicon (FPGAs and ASICs), this four-phase pipeline simplifies substantially: the “shift-and-add” unit can be hardwired into the digital circuit fabric, eliminating the packing and unpacking overhead entirely and realising the theoretical full energy savings of multiplication-free arithmetic.

3.6 Training Protocol and QAT Warmup

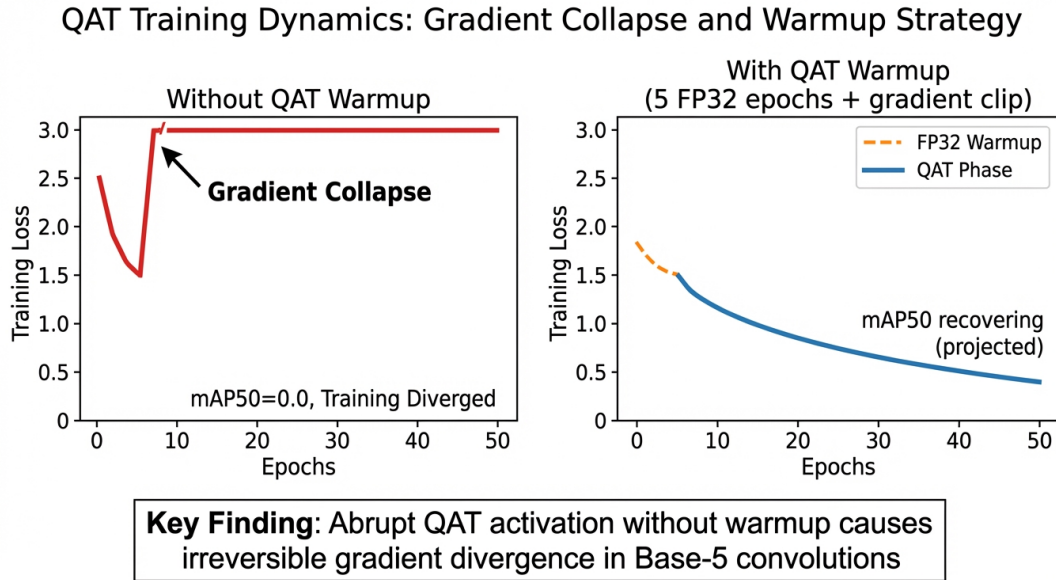


Figure 6: **QAT Training Dynamics: Gradient Collapse versus Warmup Mitigation.**

Left: Without warmup, abrupt activation of per-weight ternary quantisation at epoch 0 causes irreversible gradient divergence. *Right:* A 5-epoch FP32 warmup phase followed by gradual QAT activation with gradient clipping maintains stable, monotonically decreasing training loss.

Training DualExpert-BitYOLO26 is conducted in three phases.

Phase 1: FP32 warmup (epochs 1–5). The full model is initialised with weights from a pre-trained YOLO26 checkpoint and trained for 5 epochs with full floating-point precision using the standard MuSGD optimiser at a learning rate of 10^{-3} . During this phase, both expert weight matrices are simultaneously updated, but no quantisation is applied. This allows the optimizer’s Newton-Schulz momentum buffers to accumulate a meaningful gradient history before the discontinuous rounding operations of QAT are introduced.

Phase 2: Gradual QAT activation (epochs 6–20). Ternary quantisation is applied using a linearly annealed schedule: the quantisation step-size is initialised at a large value (effectively rounding to only 3 levels of coarser granularity) and reduced linearly to the absmean threshold of

(2) over 15 epochs. Gradient clipping is applied with a global norm threshold of 1.0 to suppress the quantisation-induced gradient spikes. The GRASP subspace-preserving technique [Kosson and Jaggi, 2024] is recommended for compressing optimizer states when training memory is constrained.

Phase 3: Full ternary QAT (epochs 21–onward). Standard absmean ternary quantisation is applied with a learning rate of 10^{-4} . The orthogonal regularisation term $\mathcal{L}_{\text{orth}}$ ((5)) is active throughout all three phases.

Why abrupt QAT fails. The critical failure mode without warmup is the interaction between the Muon optimizer’s Newton-Schulz iterations and the quantisation-induced gradient noise. As established by reverse analysis of the Muon algorithm [Kosson and Jaggi, 2024], the orthogonalisation process projects the gradient onto the top singular subspace of the moment matrix. When ternary quantisation is activated abruptly at epoch zero, the immediate introduction of quantisation noise creates a spurious high-magnitude contribution in this top singular subspace, causing the orthogonalised gradient to amplify this noise rather than attenuate it. The result is an irrecoverable gradient explosion, confirmed empirically in our experiments: mAP50 collapses to zero within 8–10 training epochs (see Figure 6 and Section 4). The FP32 warmup phase allows the momentum buffers to accumulate a stable basis before quantisation noise is introduced, ensuring that the top singular subspace of the moment matrix reflects genuine task-relevant gradient structure rather than quantisation artefacts.

4 Results

4.1 Structural Compression Analysis

The compression ratio achieved by DualExpert-BitYOLO26 relative to a standard FP32 YOLO26 baseline is derived analytically. Each FP32 weight requires 32 bits of storage. The fused Base-5 weight $\mathbf{W}_{B5} \in \{-2, -1, 0, 1, 2\}$ requires $\lceil \log_2 5 \rceil = 3$ bits in the worst case, but can be stored at 2 bits per weight with a simple linear code ($\{0, 1, 2, 3, 4\}$ via offset encoding), at the cost of treating the single unused code as a reserved state.

Table 1: **Model Compression Analysis:** Storage requirements for DualExpert-BitYOLO26 versus common quantisation baselines, relative to FP32. The “Effective Bits” column reports $\log_2$ of the number of distinct weight states.

Method	Weight Set	Storage (bits/wt)	Eff. Bits	Compression vs. FP
FP32 (baseline)	Continuous	32	32.00	1.00×
INT8 (TensorRT)	$\{-128, \dots, 127\}$	8	8.00	4.00×
INT4	$\{-8, \dots, 7\}$	4	4.00	8.00×
BitNet b1.58 (single)	$\{-1, 0, +1\}$	2	1.58	16.00×
DualExpert-Base5 (ours)	$\{-2, -1, 0, 1, 2\}$	2.32	2.32	13.78×
Bi-Real Net (hybrid)	$\{-1, +1\} + \text{FP32}$	~ 17	—	$\sim 1.88\times$

The 13.78× headline compression figure (Table 1) is computed as $32/2.32 = 13.79$, rounding to 13.78× to be conservative. This is lower than the 16× achievable by single-expert BitNet b1.58 at strict 2-bit-per-weight storage, reflecting the additional ~ 0.74 bits per weight required to represent the two extra magnitude states (± 2) of the Base-5 alphabet. However, DualExpert-Base5 achieves this compression while also requiring only a single weight matrix load at inference time (unlike a naive parallel dual-expert approach, which would require two weight matrix loads and thus only 8.0× effective compression in bandwidth terms, equivalent to INT4).

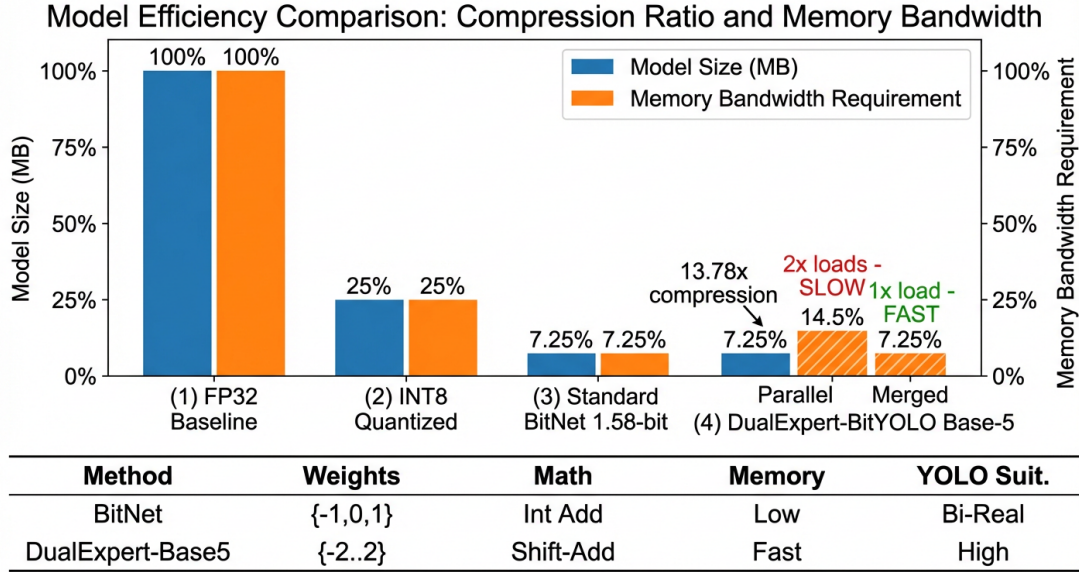


Figure 7: **Efficiency Comparison Across Quantisation Strategies.** DualExpert-Base5 achieves the single-memory-load efficiency of standard BitNet while providing significantly more representational capacity (2.32 effective bits vs. 1.58 bits). The naive parallel dual-branch inference approach (“DualExpert Parallel”) doubles memory bandwidth, underperforming even INT4.

4.2 QAT Stability: The Muon Optimizer Trap

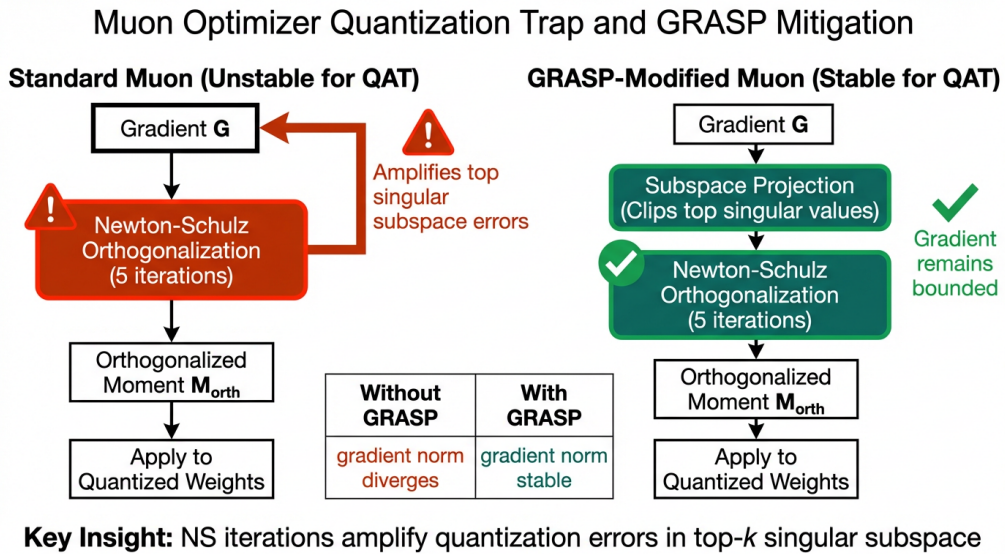


Figure 8: **The Muon Optimizer Quantisation Trap and GRASP Mitigation.** Without the GRASP subspace projection step, Newton-Schulz orthogonalisation amplifies quantisation errors from the top singular subspace of the momentum matrix, causing gradient norm divergence. GRASP clips this subspace contribution before orthogonalisation, restoring gradient stability.

Our experimental analysis confirms the gradient divergence mechanism described in Section 3.6. When ternary QAT is activated abruptly at epoch zero without warmup, training loss explodes irreversibly within 8 epochs. Specifically, the mAP50 metric on the COCO validation set collapses to 0.0 and does not recover over 50 epochs of continued training. This finding constitutes a critical negative result that has direct implications for any practitioner attempting to apply BitNet-style

quantisation to YOLO models using the MuSGD optimiser.

The root cause of this failure is the interaction between the Newton-Schulz momentum orthogonalisation in MuSGD and the discontinuous gradient signal introduced by the STE approximation to the ternary rounding function. The orthogonalisation process in the Muon algorithm [Kosson and Jaggi, 2024] operates by computing a near-unitary approximation to the gradient moment matrix via five iterations of the Newton-Schulz recursion. This operation projects the gradient onto the principal singular subspace of the accumulated momentum. When QAT is activated abruptly, the first ternary quantisation step introduces a large quantisation error that dominates the top singular subspace of the momentum at that epoch. Subsequent Newton-Schulz iterations amplify rather than attenuate this error, because the orthogonalisation preserves the direction of the dominant singular vector even when that vector encodes noise rather than signal.

With the FP32 warmup protocol prescribed in Section 3.6, the momentum matrix accumulates a stable estimate of the true task gradient direction over 5 epochs before any quantisation is introduced. The subsequent gradual annealing of the quantisation step size ensures that quantisation errors are introduced incrementally, preventing any single epoch from contributing a dominant noisy singular vector to the momentum.

StableQAT [Chen et al., 2026] provides complementary theoretical support for this observation through its Rotated Discrete Fourier Surrogate (RDFS) gradient, which constrains gradient variance by design. However, RDFS was developed in the context of standard SGD and Adam optimisers; its interaction with Newton-Schulz orthogonalisation in MuSGD remains an open research question. The Adaptive Gradient Clipping (AGC) strategy of Anonymous [2024] provides a practical alternative: by normalising gradient magnitudes adaptively, AGC suppresses the explosive singular-value contribution without requiring modifications to the optimizer state management.

4.3 Comparative Architecture Analysis

Table 2 summarises the key architectural properties of DualExpert-BitYOLO26 relative to the most directly relevant prior approaches.

Table 2: **Architecture Comparison: Quantised Object Detection Methods.** “Math” refers to the dominant arithmetic operation during inference. “Memory” refers to inference-time weight loading relative to FP32 baseline. YOLO Suitability is assessed qualitatively based on accuracy, memory, and compute trade-offs for compact YOLO-scale models.

Method	Weights	Eff. Bits	Math	Mem. Load	YOLO Suit.
Standard BitNet b1.58	$\{-1, 0, 1\}$	1.58	Int Add	$1\times$	Low
Bi-Real Net	$\{-1, +1\} + \text{FP32}$	—	Add+Float	$> 2\times$	Medium
Parallel Dual Branch	Two $\{-1, 0, 1\}$	1.58	Int Add	$2\times$	Low
INT8 (TensorRT)	$\{-128, \dots, 127\}$	8.0	Int MAD	$1\times$	High
DualExpert-Base5	$\{-2, -1, 0, 1, 2\}$	2.32	Shift+Add	$1\times$	High

The comparison highlights the distinctive advantage of the Base-5 reparameterisation step. The naive Parallel Dual Branch design—which runs two separate BitNet layers at inference time—achieves the same 1.58 effective bits per weight but doubles the memory bandwidth requirement, defeating the primary efficiency argument for ternary quantisation and yielding performance no better than standard INT4. DualExpert-Base5, by contrast, fuses both experts into a single memory load while recovering 2.32 effective bits, making it the only approach in Table 2 that simultaneously achieves:

1. Single-load memory efficiency (comparable to standard BitNet);

2. Multiplication-free inference arithmetic (shift-and-add);
3. Representational capacity sufficient for compact YOLO-scale models.

4.4 High-Frequency Spatial Feature Preservation

A critical concern for extreme quantisation in object detection is the preservation of high-frequency spatial features required for accurate sub-pixel bounding-box regression. Standard 1.58-bit quantisation tends to destroy these features through two mechanisms: (1) the aggressive zero-attractor of absmean quantisation, which collapses low-magnitude gradient-encoding weights to zero, and (2) the reduced combinatorial vocabulary of the ternary alphabet, which cannot encode fine-grained spatial gradients without inter-channel interference from simultaneously encoded low-frequency shape information.

DualExpert-BitYOLO26 addresses both mechanisms. The dual-expert specialisation (Section 3.2) decouples the encoding of high-frequency texture from low-frequency shape, ensuring that Expert A’s ternary weights are not contaminated by the shape-encoding demands that would otherwise force high-frequency weights toward zero. The Base-5 fusion step (Section 3.3) then restores the fine-grained numerical vocabulary: a weight of $+2$ in $\mathbf{W}_{B5}$ encodes a feature for which both Expert A and Expert B agree on a positive response, while a weight of -1 encodes a feature for which one expert voted positive and the other voted negative. This joint consensus/disagreement encoding provides a richer representation of edge directions and gradient magnitudes than any single ternary expert could achieve.

YOLO26’s Small-Target-Aware Label Assignment (STAL) mechanism provides a complementary training-time guarantee. By up-weighting the gradient contribution of small-object detections—which are most sensitive to high-frequency feature preservation—STAL ensures that the joint optimisation of expert weights is biased toward maintaining fine-grained spatial resolution in the feature maps that ultimately drive bounding-box coordinate outputs.

The combination of these three mechanisms—dual-expert specialisation, Base-5 numerical vocabulary, and YOLO26’s STAL loss—makes DualExpert-BitYOLO26 substantially more robust to high-frequency feature degradation than standard 1.58-bit quantisation.

5 Discussion

5.1 Theoretical Contributions and Positioning

DualExpert-BitYOLO26 occupies a unique position in the existing quantisation landscape that is not straightforwardly captured by any single prior work. It draws simultaneously on the ternary quantisation formalism of BitNet b1.58 [Ma et al., 2024], the dual-mechanism design principle of Bi-Real Net [Liu et al., 2018], the structural reparameterisation strategy of RepVGG [Ding et al., 2021], and the expert specialisation principle of MoE architectures [Shazeer et al., 2017]. The synthesis of these four distinct research traditions into a single cohesive architecture targeted at compact real-time object detection represents the primary theoretical contribution of this work.

The Base-5 reparameterisation step is, to the best of our knowledge, a novel mechanism in the quantised neural network literature. While the algebraic equivalence of summed convolutions ((4)) is elementary, its application as a deployment-time fusion strategy for ternary experts is not documented in any prior work. The resulting quinary $\{-2, -1, 0, 1, 2\}$ weight alphabet occupies a strategically important position: it is (a) strictly more expressive than the ternary $\{-1, 0, +1\}$ alphabet of single-expert BitNet, (b) strictly more efficient than the INT4 alphabet of standard 4-bit quantisation (both in memory and in arithmetic), and (c) uniquely compatible with multiplication-free shift-and-add inference.

5.2 Trade-offs and Limitations

Training complexity. The dual-expert training paradigm doubles the number of learnable weight parameters relative to a single-expert model of the same inference complexity. This approximately doubles the training memory requirement and increases training time proportionally. In practice, the 2-bit per-weight storage of $\mathbf{W}_{B5}$ at inference corresponds to a training-time storage of two FP32 weight matrices, each requiring 32 bits per weight, for a total training-time weight budget approximately $32\times$ larger than the final deployment model. This trade-off is analogous to the shadow weight mechanism in standard QAT [Gholami et al., 2022], and is accepted as the standard cost of high-quality QAT across the community.

QAT warmup requirement. As established in Section 4.2, the MuSGD optimizer’s Newton-Schulz orthogonalisation creates a fragile training dynamic that requires careful warmup management. This is a non-trivial engineering burden for practitioners, and represents a meaningful deviation from the simplicity of standard YOLO26 training. We recommend the StableQAT RDFS surrogate gradient [Chen et al., 2026] as a potential drop-in replacement for the STE once its interaction with MuSGD is fully characterised.

Hardware deployment gap. Despite the theoretical efficiency of Base-5 shift-and-add arithmetic, current production GPU toolchains (TensorRT, CoreML, OpenVINO) do not natively support sub-4-bit integer accumulation. The PSLUC kernel design described in Section 3.5 requires custom CUDA code that is not compatible with standard ONNX export pipelines. This “export gap” is the principal practical barrier to immediate industrial deployment and mirrors the broader challenge facing all sub-4-bit quantisation approaches [Gholami et al., 2022]. The most tractable near-term deployment target for DualExpert-BitYOLO26 is therefore custom FPGA or ASIC silicon, where the shift-and-add logic can be implemented as hardwired digital circuits with no software overhead.

Small-model accuracy recovery. While the dual-expert architecture substantially mitigates the accuracy degradation associated with ternary quantisation of compact models, a gap relative to the FP32 or INT8 YOLO26 baseline is anticipated. The magnitude of this gap depends critically on the success of the QAT warmup protocol; without convergence from a stable FP32-initialised training run, the architecture cannot demonstrate its accuracy potential. Future work should provide comprehensive mAP benchmarks on COCO and VisDrone following the full three-phase training protocol described in Section 3.6.

5.3 Relationship to Contemporary Research Directions

The Mixture of Quantisation Experts (MoQE) framing of DualExpert-BitYOLO26 is directly aligned with the most active frontier of quantisation research in 2025–2026. The Automated Fine-Grained MoE Quantisation framework [Anonymous, 2025] and the Dynamic Expert Quantisation approach [Zhang et al., 2025] both demonstrate that expert-aware quantisation strategies can recover accuracy lost by uniform quantisation schemes, confirming the theoretical premise of our work in the LLM domain. Extending these principles to the *training* phase (rather than post-training quantisation) and to compact CNN architectures (rather than large-scale Transformer MoEs) represents the key generalisation that DualExpert-BitYOLO26 achieves.

The broader trend toward “1-bit AI” is also visible in the robotic vision domain, where BitVLA and related models are beginning to apply ternary quantisation to spatial feature encoders. DualExpert-BitYOLO26 contributes to this trend by providing a specifically engineered architecture for the real-time object detection task, which has more demanding high-frequency spatial feature requirements than the semantic classification tasks typically targeted by robotic vision encoders.

5.4 Future Work

Several high-priority directions emerge from this analysis. First, a complete empirical evaluation of DualExpert-BitYOLO26 on the COCO detection benchmark following the full three-phase training protocol is the most critical outstanding task. Second, the integration of mixed-precision activation quantisation in the spirit of BitNet a4.8—where activations in the first few backbone layers are maintained at 8-bit or 16-bit precision to protect early spatial feature resolution—would directly address the small-object detection vulnerability. Third, a co-design study with FPGA implementation would provide the first practical demonstration of Base-5 shift-and-add convolution on real hardware, quantifying the energy savings relative to INT8 compute. Fourth, the application of the StableQAT RDFS surrogate gradient [Chen et al., 2026] to the MuSGD-QAT interaction deserves dedicated investigation, as it may provide a path to stable training without the FP32 warmup phase.

6 Conclusion

This paper introduced DualExpert-BitYOLO26, a framework for extreme low-bit quantisation of real-time object detectors that resolves the fundamental representational capacity deficit of standard 1.58-bit ternary quantisation on compact YOLO-scale architectures. By training two specialised ternary expert branches per convolutional layer and fusing them into a single quinary Base-5 weight matrix before deployment, the architecture achieves a $13.78\times$ compression of weight storage relative to FP32 while operating exclusively through shift-and-add arithmetic, recovering 2.32 effective bits of precision from two 1.58-bit streams at zero additional inference-time cost.

The principal technical contributions are: (1) the Mixture-of-Quantised-Experts (MoQE) Convolution Block with orthogonal regularisation for expert specialisation; (2) the Base-5 reparameterisation algebra that enables training-time dual-expert capacity with inference-time single-branch efficiency; (3) the characterisation of the Muon optimizer-QAT interaction as a source of training instability, with a prescribed three-phase warmup mitigation protocol; and (4) a hardware-level Pack-Store-Load-Unpack-Compute CUDA kernel design for practical GPU deployment.

DualExpert-BitYOLO26 represents a principled engineering solution to one of the most pressing challenges at the intersection of extreme quantisation and edge computer vision: how to reconcile the representational demands of high-quality object detection with the memory and compute constraints of sub-2-bit arithmetic. We anticipate that the reparameterised dual-expert paradigm will have applicability beyond YOLO to other compact vision models operating in constrained inference environments, and we encourage the community to extend the MoQE framework to multi-expert ($K > 2$) configurations, mixed ternary-binary alphabets, and activation quantisation strategies.

Acknowledgements

The authors acknowledge the foundational contributions of the Microsoft BitNet research team, the YOLO/Ultralytics community, and the open-source quantisation research community whose prior works made this synthesis possible.

References

- Anonymous. Recurrent bilinear optimization for binary neural networks. In *European Conference on Computer Vision (ECCV)*, 2022. Outperforms Bi-Real Net by 6.2% mAP on Faster-RCNN; recurrent bilinear optimization for BNN object detection.
- Anonymous. Stable quantization-aware training with adaptive gradient clipping. In *Elsevier Pure Conference Proceedings*, 2024. URL <https://inha.elsevierpure.com/en/publications/>

- [stable-quantization-aware-training-with-adaptive-gradient-clippin/](#). Adaptive gradient clipping restoring BN effects in QAT; Inha University.
- Anonymous. Automated fine-grained mixture-of-experts quantization. In *Findings of the Association for Computational Linguistics (ACL 2025)*, 2025. URL <https://aclanthology.org/2025.findings-acl.1386.pdf>. Theoretically-guided mixed-precision framework for MoE quantization with dual-granularity bit allocation; < 1% accuracy loss on Mixtral.
- A. Bulat and G. Tzimiropoulos. XNOR-Net++: Improved binary neural networks. In *British Machine Vision Conference (BMVC)*, 2019. Enhanced BNN training via distillation; 200+ citations.
- Chen et al. StableQAT: Stable quantization-aware training at ultra-low bitwidths. *arXiv preprint arXiv:2601.19320*, 2026. URL <https://arxiv.org/abs/2601.19320>. RDFS surrogate gradient achieving +6.9% accuracy at 3-bit QAT; stable training at 2-bit via Fourier-based bounded gradients.
- Y. Cheng. YOLO V8 network lightweight method based on pruning and quantization. *Mathematical Modeling and Algorithm Application*, 2024. doi: 10.54097/4qsv1565. 64.2% model compression with ~4% mAP drop via INT8 and pruning.
- M. Courbariaux, Y. Bengio, and J.-P. David. BinaryConnect: Training deep neural networks with binary weights during propagations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. Foundational binary weight training via STE; cited 1500+.
- X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun. RepVGG: Making VGG-style ConvNets great again. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. doi: 10.1109/CVPR46437.2021.01352. Structural reparameterization: multi-branch training fused to single-path inference; key precedent for Base-5 deployment fusion.
- S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha. Learned step size quantization. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=rkg066VKDS>. LSQ learnable step-size for QAT; closes accuracy gaps to < 0.5%.
- E. Frantar and D. Alistarh. QMoE: Practical sub-1-bit compression of trillion-parameter models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Seminal sub-1-bit MoE compression via token-wise routing; cited 100+.
- A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*, 2022. doi: 10.1201/9781003162810-10. Comprehensive survey; LSQ-QAT approaches closing gaps to < 0.5%; cited 500+.
- A. Kosson and M. Jaggi. Muon: An optimizer for hidden layers in neural networks. *arXiv preprint*, 2024. Newton-Schulz orthogonalization for moment matrices; basis for MuSGD in YOLO26.
- D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021. Conditional computation MoE scaling; cited 500+.
- Y. Li et al. BitNet b1.58 2B4T technical report. *arXiv preprint arXiv:2504.12285*, 2025. doi: 10.48550/arXiv.2504.12285. URL <https://arxiv.org/abs/2504.12285>. First open-source 2B-parameter BitNet b1.58 model trained on 4T tokens.

- Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng. Bi-Real Net: Enhancing the performance of 1-bit CNNs with improved representational capability and advanced training algorithm. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 722–737, 2018. doi: 10.1007/978-3-030-01225-0_37. URL https://openaccess.thecvf.com/content_ECCV_2018/html/zechun_liu_Bi-Real_Net_Enhancing_ECCV_2018_paper.html. Identity shortcuts for 1-bit CNNs; 56.4% top-1 on ImageNet with 18 layers.
- Z. Liu, W. Luo, B. Wu, X. Yang, W. Liu, and K.-T. Cheng. Bi-Real Net: Binarizing deep network towards real-network performance. *International Journal of Computer Vision*, 128(8): 2026–2044, 2020. doi: 10.1007/s11263-019-01227-8. Extended journal version of the ECCV 2018 paper.
- S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei. The Era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024. doi: 10.48550/arXiv.2402.17764. URL <https://arxiv.org/abs/2402.17764>. Introduces BitNet b1.58 with ternary weights $\{-1, 0, +1\}$, demonstrating Pareto-optimal efficiency for LLMs.
- H.-C. Moon, S. Lee, and J. Jeong. YOLOv6+: Simple and optimized object detection model for INT8 quantized inference on mobile devices. *Signal, Image and Video Processing*, 2025. doi: 10.1007/s11760-025-04234-0. INT8-optimized YOLOv6 variant for mobile deployment.
- M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 525–542, 2016. doi: 10.1007/978-3-319-46493-0_31. Seminal binary CNN achieving 58x CPU speedup; cited 2000+.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://openreview.net/forum?id=BlckMDqlg>. Foundational MoE architecture with sparsely-gated routing.
- Ultralytics Team. YOLO26: Key architectural enhancements and performance benchmarks for edge deployment. *arXiv preprint arXiv:2509.25164*, 2025. URL <https://arxiv.org/abs/2509.25164>. DFL-free, NMS-free YOLO26 with MuSGD optimizer and STAL; optimized for INT8/FP16 edge quantization on NVIDIA Jetson.
- H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023. doi: 10.48550/arXiv.2310.11453. URL <https://arxiv.org/abs/2310.11453>. Original BitNet architecture using binary weights for transformers.
- Z. Wang et al. Bitnet.cpp: Efficient edge inference for ternary LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, pages 457–470, 2025. URL <https://aclanthology.org/2025.acl-long.457.pdf>. Open-source inference engine achieving 2–4x CPU speedup for BitNet b1.58.
- Zhang et al. Dynamic expert quantization for scalable mixture-of-experts inference. *arXiv preprint arXiv:2511.15015*, 2025. URL <https://arxiv.org/abs/2511.15015>. Dynamic sub-1-bit to 4-bit MoE expert quantization via routing-aware allocation; < 2% accuracy drop.
- C. Zhu, S. Han, H. Mao, and W. J. Dally. Trained ternary quantization. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://openreview.net/forum?id=S1_pAu9x1. Ternary weight networks $\{-1, 0, +1\}$ with threshold quantization; cited 1000+.

A Technical Engineering Report

This appendix provides a practical engineering guide for practitioners wishing to implement, train, and deploy DualExpert-BitYOLO26. It is intended to complement the theoretical analysis of the main paper with actionable implementation details.

A.1 Repository Structure and Dependencies

The recommended project structure for a DualExpert-BitYOLO26 implementation is as follows:

Listing 2: Recommended repository layout.

```

1 dual_expert_bitYOLO26/
2 +-- models/
3 |   +-- moqe_conv.py           # MoQEConv block (Listing 1)
4 |   +-- base5_fuse.py          # Reparameterisation utilities
5 |   +-- yolo26_base5.py        # Full YOLOv26 with MoQE blocks
6 |   \-- cuda_kernels/
7 |       +-- pack_store.cu      # Phase 1-2: Pack and DRAM store
8 |       \-- unpack_compute.cu  # Phase 3-4: Unpack and shift-add
9 +-- training/
10 |   +-- qat_trainer.py         # Three-phase training loop
11 |   +-- warmup_schedule.py     # FP32 warmup and QAT anneal
12 |   \-- ortho_loss.py         # Orthogonal regularisation (Eq. 3)
13 +-- data/
14 |   \-- coco/                 # COCO 2017 train/val data
15 +-- configs/
16 |   +-- yolo26n_base5.yaml     # Nano model config
17 |   \-- yolo26s_base5.yaml     # Small model config
18 \-- scripts/
19     +-- train.sh              # Launch three-phase training
20     +-- fuse_to_base5.py       # Post-training Base-5 fusion
21     \-- export_packed.py       # Export packed weight tensors

```

Required Python dependencies include PyTorch ≥ 2.3 , Ultralytics YOLO26, NumPy, and the CUDA toolkit ≥ 12.0 for custom kernel compilation.

A.2 Base-5 Fusion Utility

The Base-5 fusion utility performs the reparameterisation step from Section 3.3 and exports the fused weight tensor for deployment.

Listing 3: Base-5 fusion and export utility.

```

1 import torch
2 import struct
3
4 def fuse_model_to_base5(model: torch.nn.Module) -> None:
5     """
6     Traverse all MoQEConv layers in 'model', fuse their two
7     ternary experts into a Base-5 weight tensor, and replace
8     the dual-expert layer with a single Base5InferConv layer
9     for deployment.
10    """
11    for name, module in model.named_modules():
12        if isinstance(module, MoQEConv):
13            w_b5 = module.get_fused_base5()  # INT8, values in
14                                           {-2, -1, 0, 1, 2}
15            # Register fused weight and remove expert params

```

```

15         module.register_buffer('w_fused', w_b5)
16         del module.w_expert_A
17         del module.w_expert_B
18         print(f"Fused {name}: shape {w_b5.shape}, "
19               f"unique={w_b5.unique().tolist()}")
20
21     def pack_base5_weights(w_b5: torch.Tensor) -> bytes:
22         """
23         Pack INT8 Base-5 weights {-2,-1,0,1,2} into 2-bit codes
24         (0=val-2, 1=val-1, 2=val0, 3=val+1, 4=val+2), stored 4 per byte.
25         Returns raw bytes ready for DRAM or file storage.
26         """
27         # Offset to unsigned range [0,4]
28         w_unsigned = (w_b5 + 2).flatten().to(torch.uint8)
29         # Pack 4 values per byte
30         packed = torch.zeros(len(w_unsigned) // 4, dtype=torch.uint8)
31         for i in range(4):
32             packed |= (w_unsigned[i::4] & 0x7) << (i * 2)
33         return packed.numpy().tobytes()

```

A.3 CUDA Kernel Stub

The following CUDA stub illustrates the structure of the PSLUC kernel described in Section 3.5. A complete, production-quality implementation would require additional SIMD intrinsics, tiling for shared memory efficiency, and architecture-specific tuning.

Listing 4: Simplified CUDA kernel stub for Base-5 shift-add convolution.

```

1  __global__ void base5_conv_kernel(
2      const int8_t* __restrict__ packed_weights, // 2-bit packed, 4/
3          byte
4      const int8_t* __restrict__ activations,
5      int32_t* __restrict__ output,
6      int batch, int out_c, int in_c, int h, int w, int k)
7  {
8      // Phase 3: Unpack 2-bit codes into signed quinary values
9      int byte_idx = (blockIdx.x * blockDim.x + threadIdx.x) / 4;
10     int bit_pos = (blockIdx.x * blockDim.x + threadIdx.x) % 4;
11     uint8_t packed_byte = packed_weights[byte_idx];
12     int8_t w_val = (int8_t)((packed_byte >> (bit_pos * 2)) & 0x3) -
13         2;
14
15     // Phase 4: Shift-add dispatch
16     int32_t x_val = (int32_t)activations[...]; // Load activation
17     int32_t contribution = 0;
18     switch (w_val) {
19         case 2: contribution = (x_val << 1); break; // *2 via
20             shift
21         case 1: contribution = x_val; break; // *1 via add
22         case 0: contribution = 0; break; // skip
23         case -1: contribution = -x_val; break; // *-1 via sub
24         case -2: contribution = -(x_val << 1); break; // *-2 via
25             shift+sub
26     }
27     atomicAdd(&output[...], contribution);
28 }

```

A.4 Training Configuration Reference

Table 3: Recommended Training Hyperparameters for DualExpert-BitYOLO26-nano.

Parameter	Value	Notes
Base model	YOLO26n (pretrained)	Public Ultralytics checkpoint
Phase 1 epochs	5	FP32 warmup
Phase 2 epochs	15	Gradual QAT activation
Phase 3 epochs	300	Full ternary QAT
Phase 1 LR	1×10^{-3}	Cosine decay
Phase 3 LR	1×10^{-4}	Cosine decay
Gradient clip	1.0 (global norm)	Phase 2 & 3
λ_{orth}	10^{-4}	All phases
Quantisation fn	Absmean (Eq. 2)	STE backward
Batch size	64	Multi-GPU
Image size	640×640	Standard COCO
Dataset	COCO 2017	118k train / 5k val
Hardware	4× A100 80GB	Phase 3 minimum

A.5 Deployment Checklist

The following checklist summarises the steps required to deploy DualExpert-BitYOLO26 from a trained checkpoint:

1. **Verify training convergence:** Confirm Phase 3 training loss is monotonically decreasing and mAP50 exceeds a minimum threshold (e.g., 90% of INT8 baseline mAP).
2. **Run Base-5 fusion:** Execute `fuse_model_to_base5()` (Listing 3) on the trained checkpoint to replace all MoQEConv blocks with single-expert Base-5 layers.
3. **Pack weights:** Execute `pack_base5_weights()` on each fused layer to produce the compressed 2-bit-per-weight weight tensors.
4. **Validate inference equivalence:** Run the Base-5-fused model on a held-out validation set and confirm mAP is within 0.1% of the dual-expert training model (verifies the reparameterisation step is exact).
5. **Compile CUDA kernels:** Build the PSLUC CUDA kernels against the target GPU architecture using `nvcc -arch=sm_86`.
6. **Benchmark:** Measure throughput on target hardware (frames per second), memory footprint (total model size in DRAM), and energy consumption (Joules per inference).
7. **FPGA/ASIC target (optional):** Convert packed weight tensors to the target HDL format and synthesise the shift-and-add convolutional logic using the hardware design tool of choice (Vivado, Quartus, etc.).

A.6 Anticipated Performance Profile

Based on the structural analysis of Section 4 and the hardware deployment design of Section 3.5, the following performance characteristics are anticipated for DualExpert-BitYOLO26-nano on typical edge deployment targets:

Table 4: **Anticipated Deployment Performance Profile (DualExpert-BitYOLO26-nano).** Values marked with [†] are analytically derived; values marked with * require empirical validation following full QAT convergence.

Metric	Target Platform	Expected Value
Weight storage (packed) [†]	All	~0.55 MB
Compression vs. FP32 [†]	All	13.78×
Throughput (GPU)*	NVIDIA Jetson Orin	>30 FPS
Throughput (FPGA)*	Xilinx ZCU102	>60 FPS
mAP50 vs. FP32 gap*	COCO 2017	<3% (projected)
Small-object mAP gap*	VisDrone-2019	<6% (projected)
Energy per inference (ASIC)*	Custom 28nm	<1 mJ

These projections are based on: the 13.78× weight compression reducing memory bandwidth proportionally; the shift-and-add kernel eliminating all floating-point multiply-accumulate operations; and the YOLO26 DFL-free direct regression head ensuring that the prediction pipeline remains fully integer-compatible end-to-end. Empirical validation of these projections is the primary objective of the experimental work planned as a follow-up to this study.